WEEK-11

AIM: Simulate all file allocation strategies. a. Sequential b) Indexed c) Linked.

A) SEQUENTIAL

ALGORITHM

1. **Start**
2. Initialize all disk blocks as **free**.
3. Read the number of files to be allocated.
4. For each file:
5. Read the **starting block**.
6. Read the **number of blocks required**.
7. Check whether all required blocks from the starting block are free.
8. If all required blocks are free:
9. Allocate the consecutive blocks to the file.
10. Mark those blocks as **occupied**.
11. Display the allocated blocks.
12. If any required block is already occupied:
13. Do not allocate the file.
14. Display **"File cannot be allocated"**.
15. Repeat Steps 4–7 for all files.
16. Display the status of allocated disk blocks.
17. **Stop**.

VNR VJIET

Name of the Laboratory:

_____

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

PROGRAM:

```
#include <stdio.h>
int main()
{
   int disk[100] = {0};
   int n, i, j;
   int start, length;
   printf("Enter number of files: ");
   scanf("%d", &n);
   for (i = 0; i < n; i++)
   {
      printf("\nEnter starting block for File %d: ", i + 1);
      scanf("%d", &start);
      printf("Enter number of blocks required: ");
      scanf("%d", &length);
     int free = 1;
      for (j = start; j < start + length; j++)
      {
         if (disk[j] == 1)
         {
            free = 0;
            break;     }}
      if (free == 1)
      {
        for (j = start; j < start + length; j++)
         {
            disk[j] = 1;  }
         printf("File %d allocated successfully.\n", i + 1);
         printf("Allocated blocks: ");
```

```
            for (j = start; j < start + length; j++)
            {
                printf("%d ", j);
            }
            printf("\n");
        }
        else
        {
            printf("File %d cannot be allocated.\n", i + 1);
            printf("Some blocks are already occupied.\n");
        }
    }
    printf("\nDisk Block Status:\n");

    for (i = 0; i < 100; i++)
    {
        if (disk[i] == 1)
            printf("Block %d -> Allocated\n", i);
    }
    return 0;
}
```

VNR VJIET

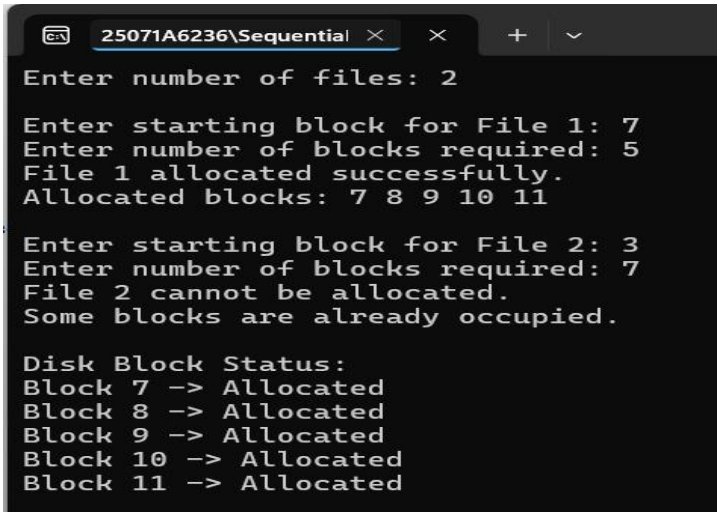
Name of the Laboratory:

Name of the Experiment:_____

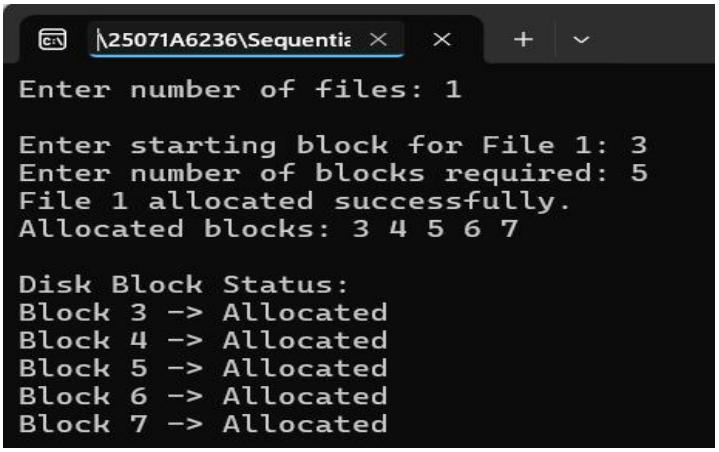
Experiment No:_____ **Date:**_____

OUTPUT:

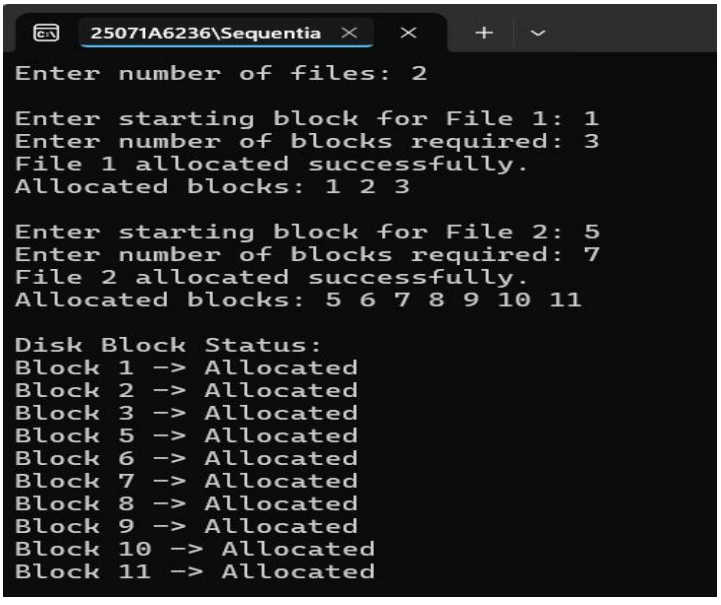
TEST CASE 1:



TEST CASE 2:



TEST CASE 3:



B) INDEXED

ALGORITHM:

1. **Start**
2. Initialize all disk blocks as **free**.
3. Read the **index block number**.
4. Check whether the index block is free.
5. If the index block is occupied, display an error and **Stop**.
6. Mark the index block as **allocated**.
7. Read the number of data blocks required by the file.
8. Read the block numbers of the required data blocks.
9. For each data block:
10. Check whether the block is free.
11. If the block is occupied, display an error and **Stop**.
12. Otherwise, allocate the block.
13. Store the addresses of all allocated data blocks in the **index block**.
14. Display the index block and the list of data blocks.
15. **Stop**.

PROGRAM:

```
#include <stdio.h>
int main()
{
int disk[100] = {0};
int indexBlock, n, i, block;
printf("Enter index block number: ");
scanf("%d", &indexBlock);
if (disk[indexBlock] == 1)
{
printf("Index block is already allocated.\n");
return 0;
}
disk[indexBlock] = 1;
printf("Enter number of data blocks required: ");
scanf("%d", &n);
int blocks[n];
printf("Enter the data block numbers:\n");
for (i = 0; i < n; i++)
{
scanf("%d", &block);
if (disk[block] == 1)
{
printf("Block %d is already allocated.\n", block);
return 0;
}
blocks[i] = block;
disk[block] = 1;
}
printf("\nFile allocated successfully.\n");
printf("Index Block: %d\n", indexBlock);
printf("Data Blocks: ");
for (i = 0; i < n; i++)
{
printf("%d ", blocks[i]);
}
printf("\n\nIndex Block %d contains addresses:\n", indexBlock);
for (i = 0; i < n; i++)
 {
printf("-> %d\n", blocks[i]);
  }
return 0;
  }
```

VNR VJIET

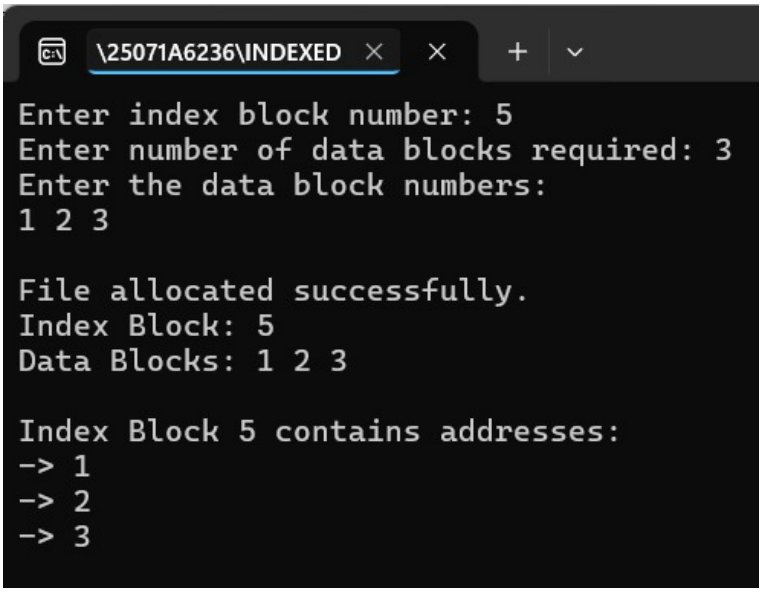
Name of the Laboratory:

Name of the Experiment:_____

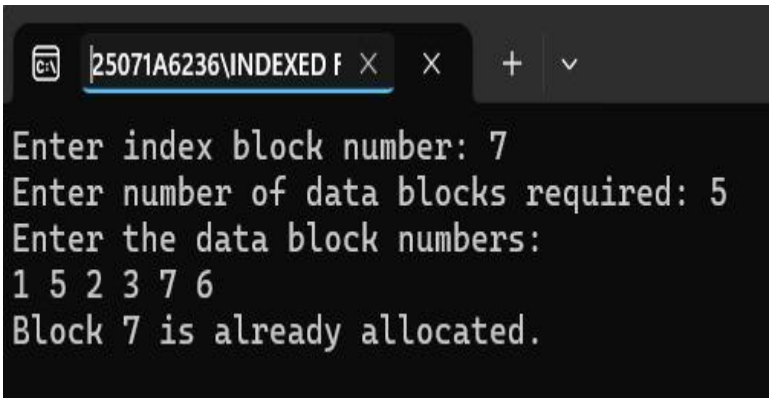
Experiment No:_____ **Date:**_____

OUTPUT:

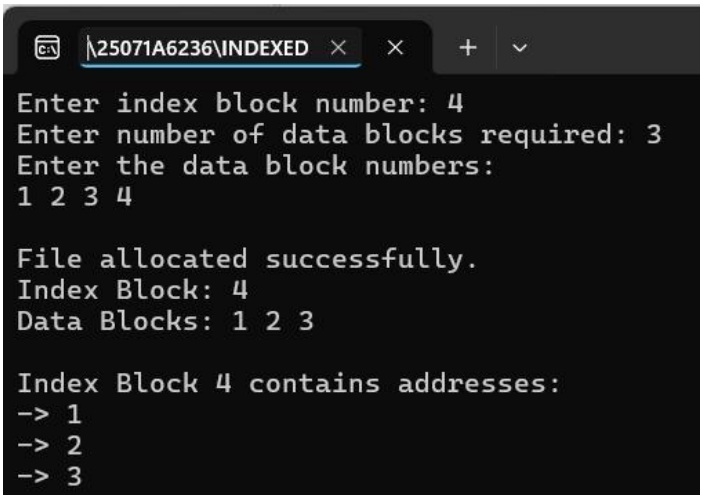
TEST CASE 1:



TEST CASE 2:



TEST CASE 3:



C) **LINKED**

ALGORITHM:

1. **Start**
2. Read the number of blocks required for the file.
3. Read the block numbers to be allocated.
4. Check whether each requested block is free.
5. If a block is already allocated:
6. Display **"Block is already allocated"**.
7. Stop the allocation.
8. If the block is free, allocate it to the file.
9. Link each allocated block to the next allocated block.
10. Set the pointer of the last block to **NULL**.
11. Display the linked list of allocated blocks.
12. **Stop**.

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

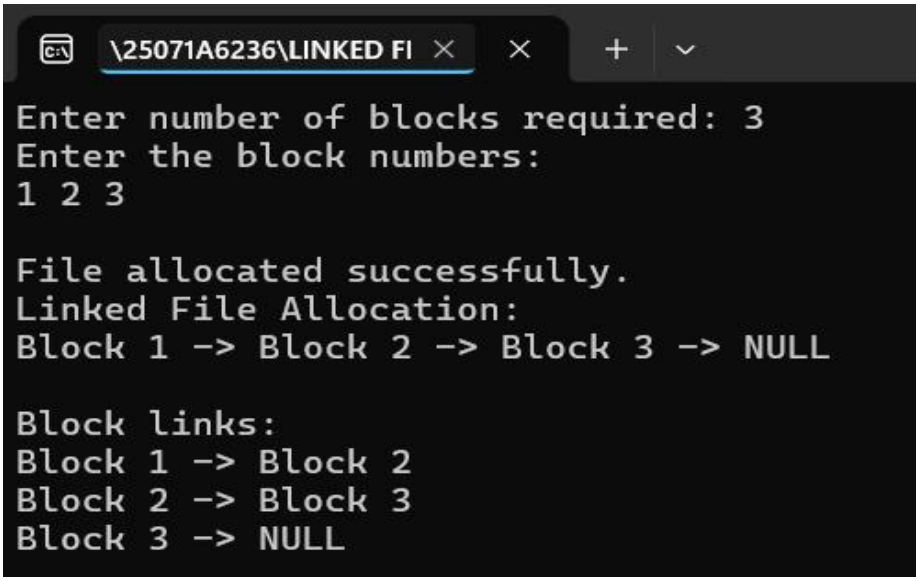
Experiment No:_____ **Date:**_____

PROGRAM:

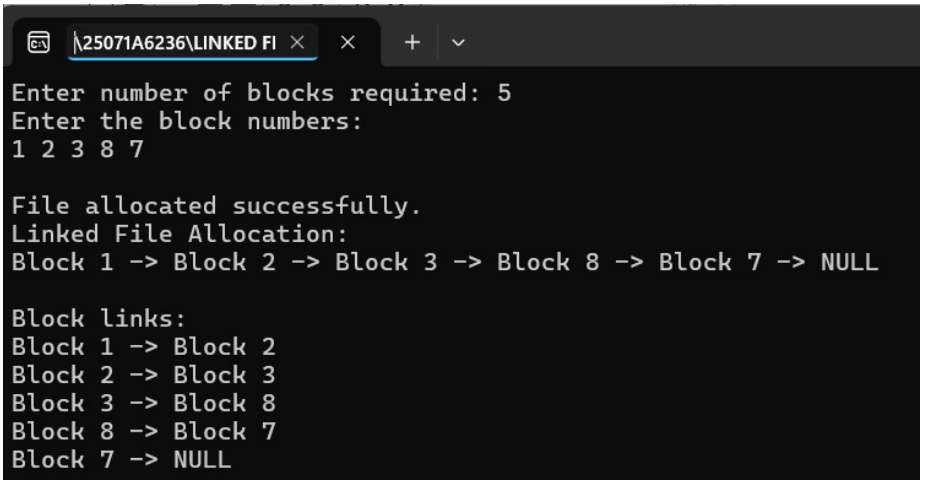
```
#include <stdio.h>
int main()
{
int disk[100] = {0};
int blocks[100];
int n, i, block;
printf("Enter number of blocks required: ");
scanf("%d", &n);
printf("Enter the block numbers:\n");
for (i = 0; i < n; i++)
{
scanf("%d", &block);
if (disk[block] == 1)  {
        printf("Block %d is already allocated.\n", block);
        return 0;   }
disk[block] = 1;
blocks[i] = block;    }
printf("\nFile allocated successfully.\n");
printf("Linked File Allocation:\n");
for (i = 0; i < n; i++)  {
printf("Block %d", blocks[i]);
if (i < n - 1){
        printf(" -> ");    }
printf(" -> NULL\n");
printf("\nBlock links:\n");
for (i = 0; i < n - 1; i++)      {
printf("Block %d -> Block %d\n",
        blocks[i], blocks[i + 1]);    }
printf("Block %d -> NULL\n", blocks[n - 1]);
return 0;
  }
```

OUTPUT:

TEST CASE 1:



TEST CASE 2:



TEST CASE 3:

